

IntentGuard: Task-Scoped Runtime Authorization for Tool-Using AI Agents

Prabhu Sivapunniyam
Independent Researcher

Abstract - Tool-using AI agents can invoke operations that are technically permitted but exceed the authority of the current user task. This mismatch matters because a relevant-looking action can disclose data, access unrelated resources, or exceed a cumulative limit. We present IntentGuard, an explicit task contract and guarded execution runtime that validate operations, resources, destinations, confirmation requirements, and action counts before invoking bounded tools. The contribution is an inspectable enforcement design and a controlled study of the incremental value of cumulative constraints; task-scoped authorization itself is established prior work. All 28 automated tests pass. On 32 synthetic cases containing 36 proposals, IntentGuard matches all expected decisions, executes all 20 authorized proposals, and executes none of the 16 proposals requiring blocking or confirmation. An operation-only control executes 11 of those 16 proposals; removing history admits one additional unauthorized action. Median batch-average predicate time ranges from 0.394 to 0.420 microseconds across resource sets of 1-10,000 entries. The measurements establish deterministic behavior and local predicate cost on constructed inputs, not general attack resistance, end-to-end agent utility, or superiority over published defenses.

Index Terms: AI agents, task authorization, prompt injection, runtime enforcement, least privilege, trajectory constraints.

I. INTRODUCTION

An AI assistant retrieving an invoice may possess credentials capable of sending messages, modifying records, and reading unrelated documents. The user may nevertheless have authorized only retrieval of one invoice amount. If retrieved content asks the assistant to forward the invoice externally, a capability check may permit the invocation while the action violates the task. We use authorization drift to describe divergence between task authority and proposed operations or cumulative effects.

Task relevance does not by itself establish authority: sending a document can help resolve an invoice question while

violating a recipient restriction. Conversely, an unfamiliar resource can be necessary for a legitimate search. An enforcement system must expose which boundaries are hard constraints and which require confirmation. Treating uncertainty as permission risks disclosure; rejecting every uncertainty can prevent useful work.

IntentGuard evaluates a normalized proposal before execution and returns ALLOW, CONFIRM, or BLOCK with a reason. A trusted tool registry declares required operands and side effects. A task lock serializes authorization, charging, and bounded execution so concurrent calls cannot independently consume the same remaining budget. The human-readable objective is stored but not interpreted semantically; contracts are supplied as structured data to separate enforcement errors from natural-language compilation errors.

The study asks what violations each policy mechanism prevents under matched proposals and what legitimate actions it obstructs. The contribution comprises an executable contract, a guarded in-memory runtime, 15 synthetic pairs, and a reproducible four-policy comparison. The results isolate resource, destination, confirmation, and count behavior. A full agent study remains necessary for adaptive attacks, ambiguous tasks, and generalization beyond the selected inputs.

II. RELATED WORK AND RESEARCH GAP

A. Evaluation and task alignment

AgentDojo [1] and InjecAgent [2] provide environments for studying indirect prompt injection and tool-using agents. Their scope is broader than the fixed proposals used here; percentages from these datasets cannot be ranked against our diagnostic counts. Task Shield [3] checks whether instructions and calls contribute to the user task. This overlaps with a proposed semantic layer, whereas the present design exposes deterministic constraints for independent inspection and replay.

B. Authorization, provenance, and trajectories

DRIFT [4] combines a planned function trajectory, parameter constraints, dynamic validation, and instruction isolation. IntentGuard implements a shared count, not equivalent plan reasoning or memory isolation. PAuth [5] derives task-scoped specifications and binds values to signed provenance; it directly addresses operator permission versus authority for a concrete operation. IntentGuard neither originates task-scoped authorization nor provides PAuth's provenance guarantee. CaMeL [6] separates control and data flows and constrains tool calls through capabilities. Explicit resource and destination sets alone cannot provide its information-flow guarantees.

These approaches already address task alignment, authorization, provenance, or trajectories. The unresolved question for this study is narrower: does cumulative enforcement add protection over an otherwise identical per-action contract while preserving authorized execution? An operation allowlist cannot distinguish resources; a per-action contract without history cannot distinguish a second permitted read from a third read exceeding a limit of two. This distinction motivates the ablation, without asserting that prior work ignores authorization drift.

III. SYSTEM MODEL AND METHODOLOGY

A. Trust boundary and representation

The model contains a user, an untrusted or fallible planner, a trusted contract authority, a guarded runtime, and bounded tools. The host owns the runtime and tool credentials. The planner supplies proposals, not contract updates, approvals, or history. Attacker-controlled tool content and planner mistakes are in scope; compromise of the host, runtime, or tool adapter is excluded. All evaluated effects are synthetic and remain in memory.

Let $C = (O, R, D, P, F, K)$ denote allowed operations, resources, destinations, prohibited operations, confirmation-required effects, and an optional count limit. A proposal $a = (o, r, d, s)$ contains operation, resource, optional

destination, and optional effect. The runtime validates operands and derives s from trusted tool metadata. Its count h records admitted invocations, including failures. Hard constraints are evaluated before confirmation; only ALLOW reaches a tool.

B. Worked task contract

Consider: "Read invoice 123. Ask before sending it to me. Do not access other documents." Listing 1 encodes this task with a two-invocation budget. The registered send adapter requires a destination and declares external_communication. Omitting that effect from the proposal cannot suppress confirmation.

Listing 1. Worked task authorization contract

```
{
  "objective": "Read invoice; confirm sending",
  "allowed_operations": ["read", "send"],
  "allowed_resources": ["invoice_123"],
  "allowed_destinations": ["user@example.com"],
  "prohibited_operations": ["delete"],
  "confirmation_required": ["external_communication"],
  "max_action_count": 2
}
```

TABLE I. WORKED CONTRACT OUTCOMES

Separate proposal	History h	Decision
Read invoice_123	0	ALLOW
Read payroll	0	BLOCK
Send to external address	0	BLOCK
Send to user@example.com	0	CONFIRM
Read invoice_123	2	BLOCK

Table I treats each row as a separate proposal. Rows other than the exhausted-budget example begin at $h = 0$; the last row begins after two admitted reads. The contract authorizes read attempts but does not automatically approve sending. A valid recipient yields CONFIRM, whereas an external recipient violates a hard boundary and yields BLOCK. CONFIRM suspends execution; authenticated approval and resumption are not implemented.

IV. ARCHITECTURE AND AUTHORIZATION ALGORITHM

A. Guarded execution boundary

Figure 1 shows the implemented path. Validated frozen dataclasses represent contracts and proposals. The host's ToolSpec registry identifies required destinations and side effects. GuardedRuntime combines this metadata with the ordered predicate and an internal action count. Registry entries are copied at initialization, and the planner cannot supply the count through the execution interface. A process boundary or external sandbox is still necessary if planner code itself is adversarial.

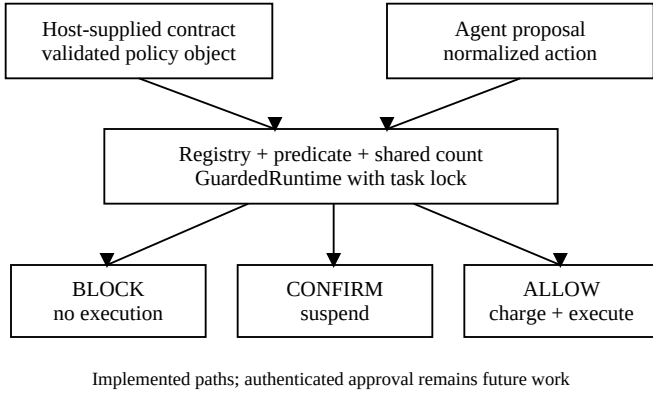


Fig. 1. Implemented execution boundary. Trusted registry metadata and a task lock mediate mock tools. CONFIRM suspends; authenticated approval and resumption remain future work.

The wrapper rejects unknown adapters, missing required destinations, unexpected destination fields, and conflicting side-effect declarations. It derives omitted effects from trusted metadata before calling the predicate. Explicitly prohibited operations take precedence over an allowlist entry. Operation, resource, destination, and count violations take precedence over confirmation. Figure 2 summarizes the resulting decision flow.

B. Execution algorithm

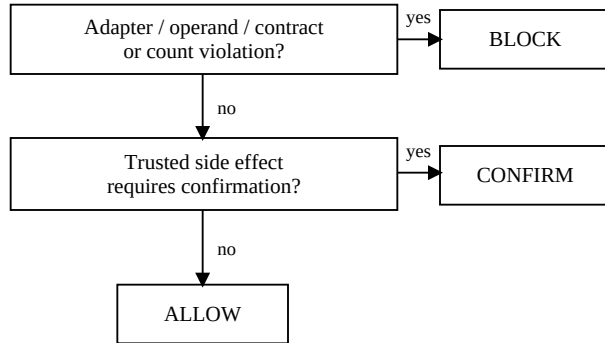


Fig. 2. Runtime decision flow after proposal validation. Missing required destinations and hard violations block before confirmation; side effects come from trusted metadata.

Algorithm 1. Guarded execution with shared history

Input: contract C, proposal a, trusted registry T
 State: internal count h; per-task lock L
 1 Validate a; reject malformed input
 2 Acquire L
 3 Check adapter and operands; derive trusted effects
 4 If adapter checks fail: decision = BLOCK
 5 Else evaluate operation, resource, destination, count, then confirmation constraints
 6 If BLOCK or CONFIRM: record; return without tool
 7 Increment h before invoking the handler
 8 Execute; retain output or exception in trace
 9 Return ALLOW with execution outcome
 On every exit from the locked region: release L

Every ALLOW is charged before its handler runs. A tool exception is retained in the result and does not refund the charge. BLOCK and CONFIRM produce trace records without tool execution or budget consumption. Malformed contracts and proposals raise validation errors before execution. The trace records decisions and outcomes, but is neither durable nor cryptographically tamper-evident.

C. Cumulative safety and concurrency

For a finite limit K and an initial count of zero, every admission requires $h < K$ and increments h before invoking a handler. Since one task lock serializes this transition, induction bounds admitted invocations by K, including failed invocations. This property assumes a trusted host and a shared runtime instance. It does not extend to independent processes with separate counters, direct tool calls that bypass the wrapper, or authenticated multi-agent delegation.

The implementation holds the lock across bounded mock execution. This simplifies accounting and prevents a check-then-execute race, but can serialize slow handlers in a deployment. A future reservation-based scheduler must preserve the same count invariant while handling cancellation, retries, and persistence. Confirmation similarly needs a trusted approval record bound to the task, operands, and contract version; a planner-supplied approval flag is not accepted.

V. EXPERIMENTAL EVALUATION

A. Cases and local controls

The benchmark contains 15 paired designs, P01-P15, covering four operation pairs, four resource pairs, four destination pairs, two confirmation pairs, and one count pair. Each pair has an authorized member and a minimally changed drift member. These 30 cases join the original benign and indirect-injection fixtures, yielding 32 cases and 36 proposals: 20 labeled ALLOW, 14 BLOCK, and two CONFIRM. Labels are authored before execution and have not been independently reviewed.

Four policies receive the same proposals and tool metadata. No enforcement removes task-policy restrictions. Operation only retains the operation allowlist but omits resource, destination, confirmation, and count restrictions. No history uses the full contract with only the count limit removed. IntentGuard uses all implemented constraints. Every condition uses the same operand-validation wrapper and mock handlers, preventing tool parsing differences from confounding this policy comparison.

Each case starts with a fresh runtime and zero charged actions. Its complete proposal sequence is replayed, including proposals following a denial. Handlers append synthetic effects to an in-memory list; no files, messages, or financial systems are changed. The original injection fixture's untrusted text is metadata, not input to an LLM. Consequently, the study evaluates fixed action proposals, not whether an attack successfully induces a model to generate them.

B. Outcomes and implementation verification

We count decisions matching the predefined labels, cases with every decision correct, and authorized versus unauthorized mock executions. A proposal labeled CONFIRM is considered unauthorized to execute before approval. Authorized execution measures whether the mock handler was called for an ALLOW-labeled proposal; it does not measure completion of a user's natural-language task. Baseline disagreements are retained as results rather than treated as runner failures.

The 28-test suite covers the original deterministic checks, input validation, destination and confirmation behavior, hard-block precedence, execution suppression, failed-call accounting, concurrency, and benchmark-runner regressions. A concurrency test submits 40 calls through eight worker threads to one runtime with a limit of two. A separate trajectory test performs three actual read proposals and checks that only the first two execute. Tests also verify that omitted side effects cannot bypass confirmation and incorrect benchmark labels cause a nonzero runner exit.

C. Predicate timing protocol

The timing experiment repeats one authorized read with prebuilt resource sets of 1, 10, 100, 1,000, and 10,000 entries. Each size receives 10,000 warm-up calls and 31 batches of 20,000 evaluations. Size order is shuffled within each round using seed 20260930. A high-resolution monotonic clock measures batch duration, divided by call count. Medians and interquartile ranges describe batch means, not individual-call tail latency.

Measurements use CPython 3.12.14 on Windows 11 build 26200 and an Intel64 Family 6 Model 189 processor, collected September 30, 2026, local time. Background activity is not isolated. Timing includes Python call and loop overhead but excludes contract construction, wrapper locking, tool metadata processing, traces, model inference, and tool execution. The 3.1 million timed evaluations are repeated measurements of one proposal, not independent security cases.

D. Reproduction and evidence retention

The standard-library runner validates cases, replays four policies, writes expected and actual decisions with mock effects, and records input and source hashes. The retained `runtime_benchmark_results.json` and `runtime_timing_results.json` are the snapshots used for this revision. The benchmark and timing scripts reproduce their respective procedures; new timing values can vary. Automated tests and the benchmark are configured to run in continuous integration. No statistical significance claim is made from these hand-authored cases or repeated calls.

VI. RESULTS AND ANALYSIS

A. Benefits observed in fixed-proposal replay

All 28 automated tests pass. IntentGuard matches all 36 decisions across 32 cases, executes all 20 authorized proposals, blocks 14 proposals, and suspends two for confirmation. In Table II, agreement counts matching decisions; Unauth. and Auth. denote unauthorized and authorized mock executions. None of the four policies reports a mock tool error.

TABLE II. DECISION AGREEMENT AND MOCK EXECUTIONS

Policy	Agreement	Unauth.	Auth.
No guard	20/36	16	20/20
Operation only	25/36	11	20/20
No history	35/36	1	20/20
IntentGuard	36/36	0	20/20

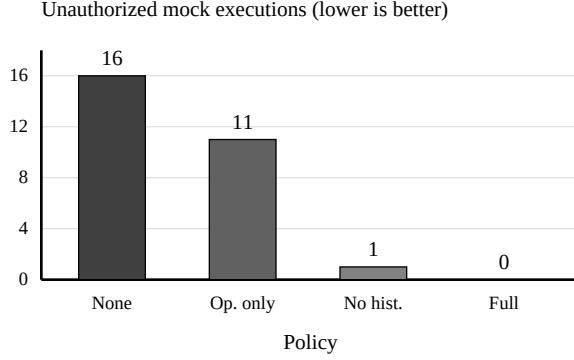


Fig. 3. Unauthorized mock executions among 16 BLOCK- or CONFIRM-labeled proposals. Full denotes IntentGuard. All policies execute the same 20 authorized proposals.

Relative to operation-only enforcement, the full contract prevents 11 additional unauthorized executions on these inputs while retaining the same 20 authorized executions. Removing history changes only the exhausted-count trajectory, admitting its third read. This isolates one incremental benefit of cumulative enforcement rather than attributing all improvement to history. The concurrency test admits two of 40 proposals submitted through eight worker threads, preserving the shared budget.

B. Resource-set scaling and local cost

Median batch-average predicate time ranges from 0.394 to 0.420 microseconds across the five sizes (Fig. 4). At 1,000 resources, the median is 0.420 microseconds and the interquartile range is 0.334-0.493 microseconds. No monotonic increase appears across the tested sizes.

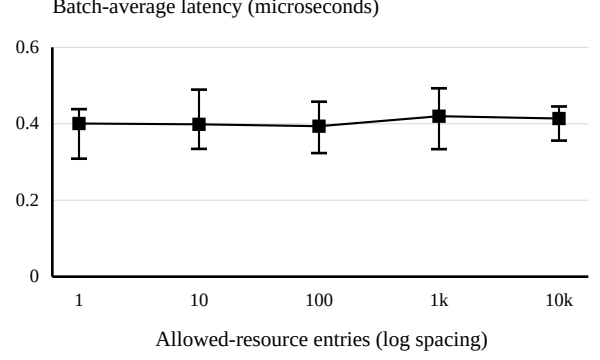


Fig. 4. Resource-set scaling of the in-memory predicate. Points are medians of 31 batch means; whiskers show the interquartile range. Construction and tool costs are excluded.

Contract construction, memory consumption, locking, tracing, and slow handlers can dominate deployment cost. They are excluded from this timing plot and must be measured separately before making a systems-performance claim.

C. Timing-estimate stability

IntentGuard has no iterative optimization or learned state, so algorithmic convergence is inapplicable. Figure 5 shows the running median of batch-average latency at 1,000 resources, ending at 0.420 microseconds after 31 batches. This describes measurement stability, not improved authorization quality with repeated calls.

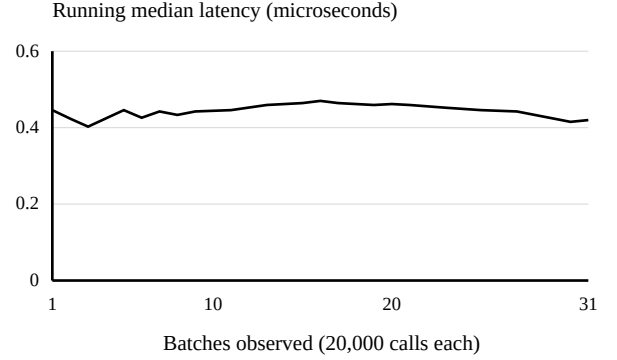


Fig. 5. Timing-estimate stability at 1,000 resources. This running-median plot is not algorithmic convergence or a learning curve.

The evidence supports specific benefits: operand-level boundaries distinguish calls sharing an operation, trusted metadata prevents omitted-effect bypass, and shared history rejects an otherwise allowed excess call. It does not establish general attack resistance or benign task completion. The next study must test whether these benefits persist on independently reviewed workloads and against stronger, faithfully reproduced defenses.

VII. LIMITATIONS

A. Evidence and construct validity

The 32 cases exercise known mechanisms and are not a random or held-out task sample. Labels have not received independent review, and no adaptive attacker or LLM generates proposals. The paired cases explain why particular predicates matter, but a perfect result does not establish population-level precision, recall, or attack success. Preserving 20 authorized mock executions is not evidence that an agent can complete 20 natural-language tasks.

The controls are deliberately limited. Operation-only enforcement cannot inspect resources or recipients, and no-history enforcement cannot track a budget. Their failures on such inputs follow from their definitions. These comparisons isolate mechanisms, not a state-of-the-art ranking. Future studies must include realistic discovery tasks where strict resource enumeration can block legitimate work, along with ambiguous requests and independently reproduced published defenses.

B. Trust and implementation boundary

Contracts are supplied by the experimenter, avoiding natural-language ambiguity and compilation errors. The guarded runtime owns a count and tool registry within one process, but does not authenticate users, persist state, isolate malicious Python code, or prevent a compromised host from invoking tools directly. Frozen dataclasses and a lock are implementation mechanisms, not cryptographic or operating-system security boundaries.

The registry defines destination requirements and effects for each supported adapter. Incorrect metadata or an adapter that uses undeclared operands can invalidate enforcement. Pure predicate calls outside the wrapper still rely on caller-supplied history and action fields. CONFIRM suspends execution but cannot currently accept and authenticate a user's approval. Durable, operand-bound approval and contract updates remain necessary for a complete interactive workflow.

C. Measurement and generalization

The microbenchmark measures one authorized path on one machine with preconstructed objects. Interpreter behavior, caches, scheduling, and background load influence its batch averages. The concurrency test verifies a count property for one scenario; it is not a concurrency performance benchmark. Broader measurements should vary action mix, misses, key lengths, policy construction, logging, persistence, and handler latency, reporting tail and end-to-end costs separately.

No semantic judge, provenance system, aggregate-value budget, disclosure-composition policy, or authenticated

delegation mechanism is evaluated. The remaining P16-P20 designs require these additional capabilities. The system therefore cannot claim protection against general information-flow attacks, cross-agent authority laundering, or arbitrary multi-step prompt injection. Statistical claims require appropriately sampled tasks and a justified analysis, not additional repetitions of fixed deterministic inputs.

VIII. CONCLUSION AND FUTURE WORK

IntentGuard exposes task authority as an executable contract and mediates bounded tools through trusted metadata and shared history. The prototype passes 28 tests and matches 36/36 decisions on 32 synthetic cases, with 20/20 authorized mock executions and zero unauthorized executions. Operation-only enforcement executes 11 unauthorized proposals; removing history admits one excess read. These differences demonstrate the intended contribution of explicit operands and cumulative policy on the tested inputs.

The next research phase should independently review labels, implement the remaining trajectory and provenance cases, add authenticated approval, and evaluate model-generated proposals under matched attack budgets. Contract compilation must be measured separately from enforcement. Faithful task-alignment and authorization baselines should be compared using unauthorized execution, benign completion, confirmation burden, and end-to-end cost together. If the incremental benefit disappears under these conditions, the contribution should be narrowed to the implementation and reproducible diagnostic artifact.

REFERENCES

- [1] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramer, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. doi: 10.52202/079017-2636.
- [2] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of ACL*, 2024, pp. 10471-10506. doi: 10.18653/v1/2024.findings-acl.624.
- [3] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," in *Proc. ACL*, 2025, pp. 29680-29697. doi: 10.18653/v1/2025.acl-long.1435.
- [4] H. Li, X. Liu, C. Chiu, D. Li, N. Zhang, and C. Xiao, "DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents," in *Advances in Neural Information Processing Systems*, vol. 38, 2025. doi: 10.52202/085713-2791.
- [5] R. K. Sharma, L. Jiang, S. Chen, and Z. Lin, "Beyond OAuth: Task-Scoped Authorization for AI Agents via Natural Language Slices," *arXiv:2603.17170v2*, Aug. 2026. doi: 10.48550/arXiv.2603.17170.
- [6] E. Debenedetti et al., "Defeating Prompt Injections by Design," *arXiv:2503.18813v2*, Jun. 2025. doi: 10.48550/arXiv.2503.18813.